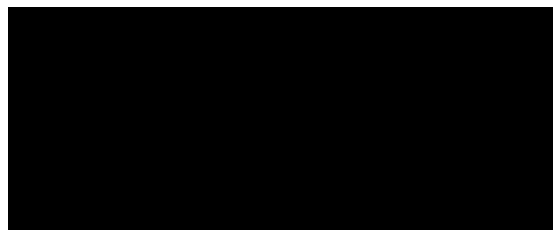


University of Waterloo
Faculty of Engineering
Department of Electrical and Computer Engineering

Insightful™

(Crowdsourced Insights Collection Platform)
Group: 2017.012

Prepared by:



Consultant:
Marc Hurwitz

30 June 2016

This report earned an overall score of 63.5 / 80

Table of Contents

Table of Contents	1
1 High Level Description of Project	3
1.1 Motivation	3
1.2 Project Objective	3
1.3 Block Diagram	5
1.3.1 Data Flow Diagram	5
1.3.2 Insightful Submodule Block Diagram	6
2 Project Specifications	7
2.1 Functional Specifications	7
2.2 Non-Functional Specifications	8
3 Detailed Design	10
3.1 Website	10
3.1.1 Selecting a Front-end Framework	10
3.1.2 TypeScript vs. JavaScript	10
3.1.3 Web Client Architecture	10
3.2 Web Server	11
3.2.1 Problem Definition	11
3.2.2 Web Server Application Framework	11
3.2.3 Authentication Strategy	12
3.2.4 Web Server Application Architecture	12
3.3 Database Design	13
3.3.1 Problem Definition	13
3.3.2 SQL (relational database) vs. NoSQL	13
3.3.3 MySQL vs. PostgreSQL	14

<u>3.3.4 Database Design and Schema</u>	15
<u>3.3.5 Database Caching, Partitioning, and Archiving</u>	16
<u>3.4 Mobile Application Programming Interface (API) Design</u>	17
<u>3.4.1 API Architecture</u>	17
<u>3.4.2 Task Serving Load Balancing</u>	17
<u>3.4.3 Optimizing Load Balancing using a Monte Carlo Simulation</u>	18
<u>3.4.4 Verifying Load Balancing Optimization using Simulated Annealing</u>	19
<u>3.5 Mobile Software Development Kit Design</u>	20
<u>3.5.1 Modularization in Software</u>	20
<u>3.5.2 Libraries vs. Frameworks</u>	21
<u>3.5.3 Dependency Managers: CocoaPods vs. Carthage</u>	22
<u>3.5.4 Architecture of the Insightful Framework</u>	22
<u>4 Discussion and Project Timeline</u>	23
<u>4.1 Evaluation of Final Design</u>	23
<u>4.2 Use of Advanced Knowledge</u>	24
<u>4.3 Creativity, Novelty, Elegance</u>	25
<u>4.4 Student Hours</u>	25
<u>4.5 Potential Safety Hazards</u>	26
<u>4.6 Project Timeline</u>	26
<u>5 Technical Terminology</u>	28
<u>6 References</u>	31

1 High Level Description of Project

1.1 Motivation

Crowdsourcing platforms enable individuals to coordinate the use of human intelligence to solve tasks that computers are unable to do. Tasks that currently benefit from human intelligence include processing of photos and videos, data verification, information collection, and general data processing. Workers of the leading solution to this problem, Amazon Mechanical Turk, complete tasks for less than half the U.S. minimum wage. Among these individuals, 70% are women and a majority of the workers reside in the United States and India [1]. Due to this, respondents are less performant with tasks that require precisely defined populations or a representative sample of the population as a whole [2]. This lack of diversity results in a dataset that is misrepresentative when conducting studies that have social science and demographic constraints.

At the same time, overwhelming majorities in both developed and less advanced nations have reported to owning some form of a smartphone. In 2015, advanced economies reported a median of 68% in smartphone ownership. This number is 37% for emerging and developing economies [3]. In other words, smartphone users are a much stronger representative sample of the world's population overall. Individuals with access to these smartphones find themselves periodically placed in front of mobile ads with over 30 billion being served every day [4]. How can something like mobile advertisements that are already interacting with the general population be leveraged to enable researchers and respondents to coordinate in solving questions posed about the world?

1.2 Project Objective

The objective of this project is to provide a platform for researchers to gather information from a diverse audience of respondents by breaking down difficult problems into smaller, collective tasks and presenting these tasks on their smartphones. This process is accomplished in three steps. First is providing researchers with the ability to create and submit crowdsourcing tasks onto the platform through a website. Each task is associated with a cost to the requester depending on the type of task. Second is the mobile **Software Development Kit** (SDK) provided to mobile app developers. The mobile SDK allows app developers to easily integrate the platform into their application **natively** and developers are incentivised through payment: for each task completed, the developer will receive a portion of the task requester's payment. Lastly, respondents as mobile app users complete these tasks and the results are collected and sent back to the requester. App users participate because they get access to premium application content as set by the application developers. Instead of showing a mobile advertisement, the application developer can make a task appear. This task, for example, may

involve selecting from a grid of dresses the one that looks the best to the viewer. This creates a source of valuable consumer data which researchers can leverage to design new products or supply local retailers. Another example could be to ask app users to tap on a foreign object in an image. This image would actually be a piece of a satellite image over a large crash site over the ocean. By breaking this large problem into a smaller data tagging task, individual contributors are collectively aiding in finding a lost airplane.

The expected benefits over the existing alternatives such as Mechanical Turk and Google Consumer Surveys include providing a crowdsourcing solution to a new and diverse range of respondents: mobile app users. Insightful is a platform that excels in creating insightful surveys and obtaining intelligence such as data labeling for research purposes. Another objective is providing a superior user experience over traditional ads. By showing a task in place of an ad, it allows the gathering of data in a controlled manner that serves the tasks in a seamless and less-intrusive way. This is in contrast to the commonly used method of linking a user to a third party website where they are taken out of context which may lead to an unsatisfactory viewing experience.

1.3 Block Diagram

1.3.1 Data Flow Diagram

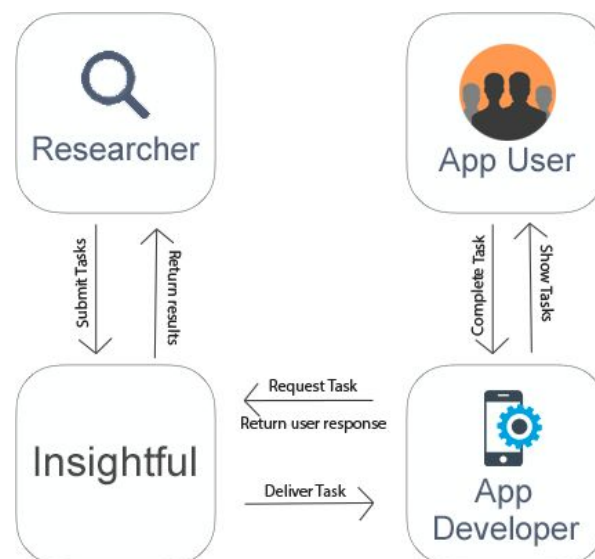


Figure 1. Block diagram illustrating the flow of data in Insightful.

Figure 1 shows the flow of data between Insightful and the three effective users of Insightful. The researcher starts by submitting a task onto Insightful that they wish to be completed. Meanwhile, the app developer creates an application that will request for a task. Insightful then delivers a task created by the researcher to the application. Once received, the app displays the task to the application user, who then completes the task in exchange for an in-app incentive.

The completed task result is then delivered back to the app made by the developer. At this point, the app returns the app user's response back to Insightful. Once enough responses are collected by Insightful to conclude a result for the task, the result is returned back to the researcher. Only the Insightful module is being completely designed.

1.3.2 Insightful Submodule Block Diagram

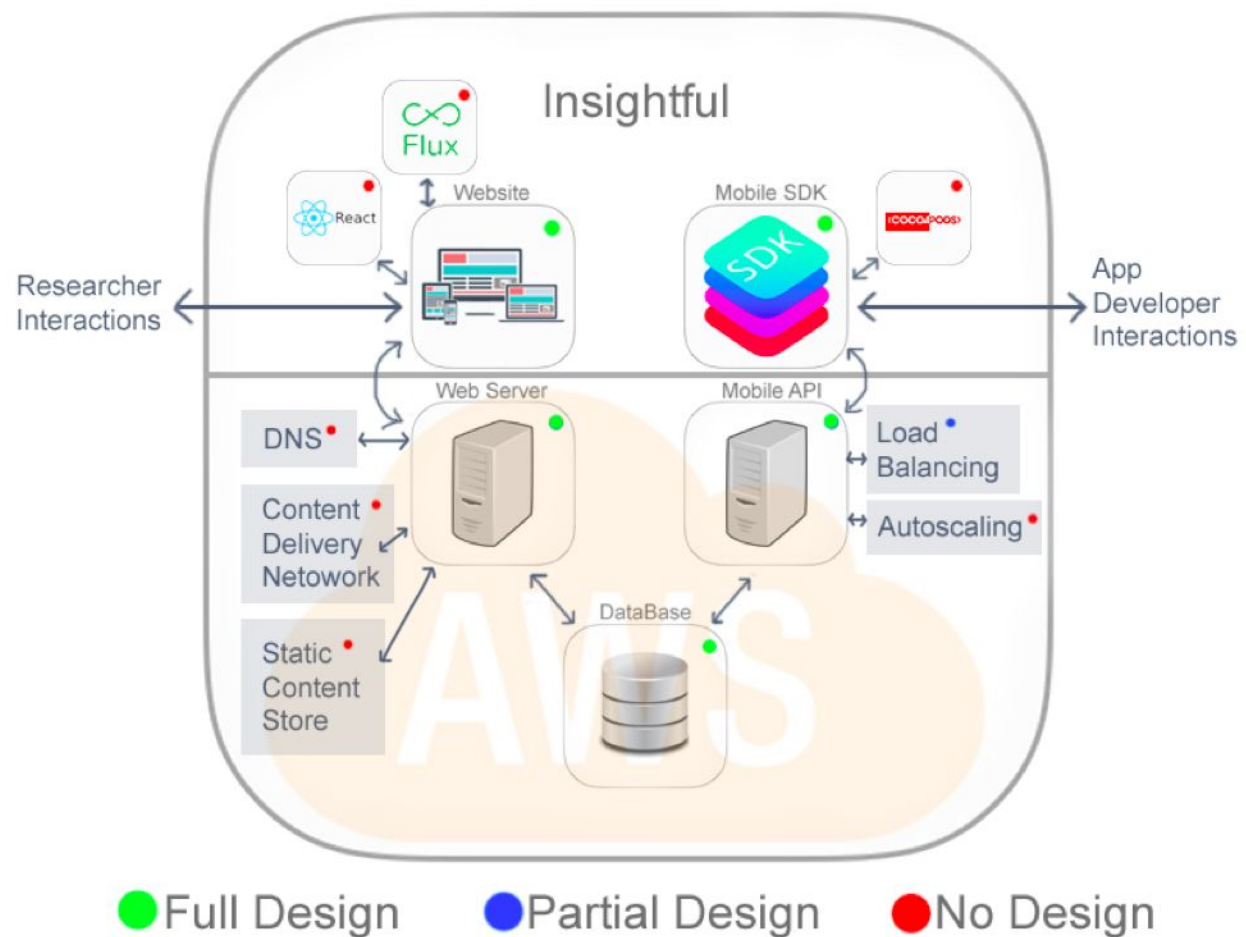


Figure 2. Block diagram illustrating the submodules of Insightful.

The project is divided into five major components: the website (front-end web client), the web server, the mobile SDK, the mobile **application programming interface** (API), and lastly the database. These components require full design and act as the foundation of Insightful. The website handles the interface between researchers and the platform. Meanwhile, the mobile SDK acts as the connecting piece between app developers and Insightful. The remaining three components: web server, mobile API, and database are built on top of Amazon's suite of web services. Load balancing and autoscaling infrastructure modules provided by Amazon enable the architecture to scale horizontally by creating replicated instances when traffic is high.

Domain name system (DNS) resolution allows users to reach the site. A content delivery network enables high availability and fast loading times by distributing deployment. The static

content store allows large assets to be used to store tasks that require large static content such as images or videos. Each major component of this diagram is built on current third-party open source libraries.

The website is built using the front-end web frameworks **React** and **Flux**. These frameworks enable developers to build fluid and interactive experiences online. The web servers communicate with the database module, where data consisting of information relating to user accounts, tasks, and payments is stored. App developers integrate Insightful into their application via the mobile SDK. The mobile SDK is distributed using **CocoaPods** and comes packaged with user-interface (UI) elements and handles all the necessary calls to the mobile API. The mobile API communicates with the database to persist task results, which can then be propagated up to the researcher through the web server and website.

2 Project Specifications

2.1 Functional Specifications

Fourteen essential functional specifications and two non-essential functional specifications of the overall high level system can be found in Table 1.

Table 1. Functional specifications of the system.

Sub-system	Description	Additional Details	Necessity
Mobile SDK	The mobile SDK must provide developers the ability to request a task to be served to their application.	It must provide developers with a minimum of 3 different visual template options to display tasks in.	Essential
Mobile SDK	The mobile SDK must be easily integratable into an existing iOS application.	The easier it is to use a mobile SDK, the more likely it is for developers to adopt it. Consequently, the majority of popular mobile SDKs require as little code as possible to be integrated into an app. As a result, it should take no more than 5 lines of code for developers to integrate the mobile SDK into an iOS app.	Essential
Mobile SDK	The mobile SDK must be resistant to malicious developers.	It must be compiled code, provide a read-only interface, and verify the authentication of the developer every time the mobile SDK is invoked.	Essential
Mobile SDK	Served tasks must be unobtrusive.	A task must be completable by an application user with no more than 2 interactions.	Essential
Mobile SDK	The mobile SDK should allow developers to theme a served task to match their application.	Theming a served task to match the application provides a more pleasant and seamless user experience when compared to conventional mobile ads.	Non-Essential

API	An API must be provided by a backend server that communicates with the mobile SDK, database, and website.	The API must act as a the method of communication between all subsystems and as the pipeline for all generated data.	Essential
API	The API must provide endpoints that serve tasks and receive information from served tasks.		Essential
API	The API must not re-serve the same task to a device more than once within the time to live (TTL) specified by the corresponding researcher.	This specification is required in order to protect the integrity of the data generated by the tasks a researcher has created (serving the same task to the same user multiple times within a short duration will result in potentially inconsistent interactions).	Essential
Database	The database must store all data generated by the mobile SDK, API, and website without any duplication of information.	It must store all generated data and transaction information since the data may be valuable in the future.	Essential
Website	The website must let researchers create an account.	For simplicity, there must be no more than 2 separate pages in the researcher sign-up flow.	Essential
Website	The website must allow researchers to create tasks to be served by the API.	It must take no longer than 24 hours for a task to be served after it has been created.	Essential
Website	The website must allow researchers to edit and delete tasks after they have been created.		Essential
Website	The website must allow researchers to download the data generated by the tasks they created.		Essential
Website	The website must be resistant to malicious researchers.	It must provide a read only interface to create and edit tasks and perform file type input validation on uploaded images.	Essential
Website	The website must provide an interface for payment.	It must be capable of paying developers and accepting payment from researchers.	Essential
Website	The website should provide data visualization and statistics to researchers for the data generated by tasks.	Statistics such as mean, standard deviation, confidence interval, p-value, regression coefficients etc. should be provided.	Non-Essential

2.2 Non-Functional Specifications

Ten essential non-functional specifications and one non-essential non-functional specification of the overall high level system can be found in Table 2.

Table 2. Non-functional specifications of the system.

Sub-system	Description	Additional Details	Necessity
Mobile SDK	The mobile SDK must be responsive.	It must be able to serve a task into a space on the mobile device's screen greater than or equal to 400 px by 300 px.	Essential
Mobile SDK	The mobile SDK must support iOS 9.0 and higher.		Essential
Mobile SDK	The mobile SDK should support iOS 8.0		Non-Essential
API	The API must be efficient with data consumption.	The size of an API response message must not exceed 10 kB.	Essential
API	The API must be able to serve tasks concurrently up to a limit.	It must be able to serve up to 10 tasks per minute per unique device ID.	Essential
API	The API must be scalable.	It must be able to support a minimum of 1000 researchers and developers using the platform.	Essential
API	The API must be able to manage a large number of servable tasks.	A researcher may have up to 100 unique, active, and servable tasks at a time.	Essential
API	The API must serve tasks quickly.	The time it takes to serve a task to an app user connected to a 4G mobile network or 10 Mbps or faster wifi network must not exceed 2 seconds.	Essential
API / Website	The API and website must experience as little downtime as possible.	They must be operating and accessible 99% of the time.	Essential
Website	The website must be responsive.	It must be usable for any screen resolution greater than or equal to 800 px by 600 px.	Essential
Website	The website must limit the size of image uploads.	It must allow researchers to upload images (for a task) with a maximum size of 1 MB.	Essential

3 Detailed Design

3.1 Website

3.1.1 Selecting a Front-end Framework

Developers rely on front-end frameworks for the structure and organization of a **JavaScript** web application. The **model-view-controller** (MVC) design pattern is a software architectural pattern for implementing user interfaces [5]. It does this by representing a running application in three interconnected parts. This separates internal structures and models from the way it is present to the user. The model manages data logic of the application. The view reflects a realization of information in any possible form. The controller reacts to changes and manipulates the model or view accordingly.

React is a JavaScript library for building user interfaces. With respect to MVC, React is considered to offer unparalleled performance advantages to the view component when compared to other popular frameworks [6]. Related React libraries focus on the Model and Controller components to complete the MVC design pattern. React's most notable feature is in its virtual **document object model** (DOM). The DOM is a cross platform language independent convention for representation and manipulation of objects in **hypertext markup language** (HTML) documents. React abstracts away from the DOM giving a simple programming abstract which empowers developers to create highly performant applications as the DOM updates only the changes that can be observed from the virtual DOM. React is based on a one-way reactive flow of data which reduces the boilerplate convention and plays with simplicity in data binding [7].

3.1.2 TypeScript vs. JavaScript

TypeScript is a superset of JavaScript which provides optional static typing, classes and interfaces. This enables JavaScript developers to use highly-productive development tools and practices like static checking and code refactoring when developing applications [8]. Type annotations make it possible to statically verify the code which allows more checks to be made by type checking implementations. This lets software components surface insights into the behaviour of internal functions. Due to these advantages offered by adopting TypeScript over traditional Javascript, It is the programming language of choice when developing the client facing website.

3.1.3 Web Client Architecture

As introduced in Section 3.1.1, the React library powers the view component of the client application. React is used alongside the **React Router** library which introduces a

synchronization component between the user interface and the **uniform resource locator** (URL) allowing for smooth transitions between subsequent pieces of the application. The React **Redux** library is used in conjunction with React as a deterministic state container. This allows developers to write applications that behave consistently and are able to better manage state transitions [9]. All of this is written alongside Typescript, as explored in Section 3.1.2, which allows for improved code clarity and an increase in developer productivity.

This architecture is put to use in creating the four main client facing components. The first component is a landing page. The second component is in the user registration flow, which allows new users to create user profiles on the insightful platform. The third component is the dashboard which allows users to create new experiments and see the results of existing ones in real-time. All these components are build with TypeScript and themed with **cascading style sheets** (CSS).

3.2 Web Server

3.2.1 Problem Definition

The web server exists as a backend component for the website. The main purpose of the web server is to serve the user facing website component to the user. The web server is built so that it is able to retrieve information from the database, provide a method of authentication, and allow users to create and populate new tasks. All of this is built in consideration to automatically scaling to a potentially large amount of traffic.

3.2.2 Web Server Application Framework

In choosing a framework to serve the web client, one can consider several available solutions such as a PHP based framework named Laravel, a Java framework called TomCat, Ruby on Rails (a framework built on top of Ruby), a Python solution called Django, and a JavaScript framework called Node.js. All of these frameworks have gained popularity among developers and all of them would work when considering the needs of the Insightful web server. However, in the end, Node.js has been chosen for this project because it offers distinct advantages over the other options. By using a JavaScript based framework, it allows the web client and webserver to be written in one programming language which ultimately boosts developer efficiency. Node.js is also one of the only frameworks offering a lightweight approach to building robust web applications. It does this by offering a powerful API though a lightweight abstraction. This allows the web service to be written in fewer lines of code while remaining highly performant. Another advantage to this framework is that it is able to processes incoming requests asynchronously. By doing this, user request are completed as soon as they can without blocking the application thread. This trade off leads to faster response rates for simultaneous requests. As Insightful draws parallels to high traffic applications such as advertising platforms [10], this asynchronous design allows the platform to better handle heavy traffic.

3.2.3 Authentication Strategy

Authentication and user management is an essential part of the Insightful platform. The server offers a gateway to multiple authentication strategies. In designing this feature, one can consider existing authentication middleware built for Node.js. **Passport.js** allows one to easily plug authentication to a Node.js application. Common strategies that are available from the Passport framework are Facebook and Google authentication, but also including local username/password authentication and **Open Authentication** (OAuth) schemes. Passport is built with session management which includes sterilization of an authentication user, managing the user session, and logging the user out [11]. In addition, this framework is designed for easy authentication without user management implementation. This leaves space to implement and maintain a custom user infrastructure [11].

On the other hand, it is also possible to build an authentication scheme from scratch. This allows the implementation to utilize an existing database to store users and a hashing algorithm to generate passwords using existing hashing functions such as bcrypt or scrypt. After deciding on the supporting infrastructure, user management and authentication must be built out. However this is tedious and requires in depth consideration into account creation, account authentication, account management, integration with social providers through OAuth, security, and replication [12]. The largest challenge in creating a robust system, is that it must well maintained. The advantage to this solution is that there is total control over the choice of infrastructure.

Due to the constraints of the Insightful platform, there is a desire for aversion from creating in house authentication schemes. In leveraging a framework such as Passport.js, an effective form of authentication can be added without added design challenges faced with custom authentication. With this approach, more development efforts can be spent in creating the product. Since Passport offers interoperability with existing user models, it can be easily integrated with the proposed user design as specified in Section 3.3 Database Design.

3.2.4 Web Server Application Architecture

User's requests for the website are routed by a DNS service. Static content, such as stylesheets for the website as described in Section 3.1, is delivered by Amazon **CloudFront**, which allows for requests to be routed to the nearest geological location, so that content can be delivered with lower latency. Other resources and static content needed by the client is stored on Amazon's **Simple Storage Service** (S3). This acts as a durable storage infrastructure to store large data files such as raw image assets and user task logs. Requests are handled by Elastic Load Balancing which distributes traffic among multiple **Elastic Compute Cloud** (EC2) instances. EC2 instances are **Unix** machines that run web application code. By having these instances located across multiple geological zones, overall fault tolerance is improved. The servers are deployed in an **Auto Scaling** group that automatically adjusts load capacities with respect to real-time traffic. This ensures that there is an optimal number of instances to

seamlessly handle demand spikes, thus maintaining good performance while minimizing costs. By employing this set of Amazon web services, one is able to account for scalability without implicitly designing infrastructure to do so. It is thus possible to horizontally scale this application out of the box when the web server is designed with these components as outlined. This in summary creates a highly available, reliable, scalable, and high performance infrastructure required for web applications while enabling an elastic, scale up and scale down infrastructure. [13]

3.3 Database Design

3.3.1 Problem Definition

Insightful generates a lot of data which must be held onto for varying amounts of time in order to serve researchers, application developers, and mobile users. Consequently, a database is required as an essential component to the platform. Below is an analysis describing the design decisions made in order to formulate the database design best suited to this platform.

3.3.2 SQL (relational database) vs. NoSQL

As of today there are generally two forms of representing data within database management systems. The first is a **Relational Database Management System (RDBMS)**, a system based on the relational model introduced by E. F. Codd. Information is stored in entries that correspond to rows in a table, with different columns of the table representing different aspects of the information being stored [14]. Relations can be thought of as mathematical sets, each of which contain certain attributes that collectively represent the information. **Structured Query Language (SQL)**, a special purpose programming language for RDBMS databases, is used to access and modify RDBMS data. Since this is a traditional system, which has already been widely adopted by industry, there exist a wide array of SQL servers that implement these principles and allow for easy usage for most users who wish to use a RDBMS database [15].

The alternative system, commonly falling under the umbrella term **non relational SQL (NoSQL)** refers loosely to all other implementations of database management systems that do not have a relational component in tabular form as the one mentioned above. There are many variations of NoSQL databases, including using key-value pairs, object oriented designs, or even graph data structures as database implementations.

An RDBMS database typically uses a fixed schema, which means information cannot be entered into a database and then changed to contain new types of information (since the table and column formats need to be changed prior to this migration). On the other hand, NoSQL databases use dynamic schemas, which means information can be added when needed and the additional data being stored need not comply with the existing schema [16]. NoSQL databases also typically scale horizontally, enabling users to simply add new servers to

increase the load capacity, whereas RDBMS databases scale vertically, requiring an increase in resources such as CPU or RAM on existing servers to handle more load. NoSQL is generally faster for simple queries, whereas RDBMS is generally faster and more robust for more complex queries. This speed and scalability of NoSQL typically comes at the expense of certain properties such as the **atomic, consistent, isolation, and durability** (ACID) properties, which RDBMS databases implement very well [16] [17].

The RDBMS model was chosen for this project due to its robustness and reliability. In order to reach the spec requirement of being accessible 99% of the time and being able to support multithreaded access (which is required if 10 tasks are to be served per unique device per minute) it is necessary to have a reliable system that implements ACID properties and is able to serve large queries. The queries from the API will mostly be complex and compound, requiring many joins across many different blocks of data to retrieve all the required information to serve a task or display analytics. There is also an archive system in place to mitigate scalability issues. Thus, the trade offs between using RDBMS and NoSQL are worth it.

3.3.3 MySQL vs. PostgreSQL

Within RDBMS databases lie several RDBMS server options one can choose from. MySQL is the most popular large-scale RDBMS database server. This open source system contains many features and there is a plethora of documentation and resources as well as third party integration tools and add-ons that support it. It is very secure, relatively scalable as an RDBMS, and performant as it gives up certain less critical SQL standards as a trade off for higher performance. It is programmable in that it allows often-executed queries to be placed in functions called a **stored procedure** and executed repeatedly. However, it does have functionality limitations because of its omission of certain SQL standards and it may not be as reliable as other RDBMS databases when it comes to certain advanced features such as references, auditing, and transactions [18].

PostgreSQL is an advanced large-scale RDBMS database server, that aims to adopt all of the SQL standards. PostgreSQL is different from other RDBMS database servers in that it implements very powerful technology to address the ACID properties, and supports concurrency 100% without sacrificing performance. This is done by foregoing read locks and using the **multi-version concurrency control** (MVCC) instead, which also ensures ACID compliance. PostgreSQL, like MySQL, is also very programmable and supports stored procedures. In addition, although both MySQL and PostgreSQL offer the trigger feature (which enables certain user-defined procedures to be run when a certain condition is met in the database), PostgreSQL offers additional advanced trigger features that MySQL does not support. The downside is that PostgreSQL is not as popular as MySQL and as a result may not have as much supporting documentation for new developers [18].

Using the above information as a foundation for choosing a technology, PostgreSQL is the perfect candidate for the platform. It ensures reliability and correctness, which are critical

technical specifications and form the foundation from which the mobile API will serve information to users. The use of MVCC boosts performance and reliability of concurrent access, which is required if 10 tasks are to be served per minute per unique device. Since there are only 100 active unique tasks per researcher as defined in the specs, there exists the chance that a task will be accessed simultaneously by different devices. Consequently, concurrency could become a potential performance bottleneck. The existence of the stored procedures enables automated scripts to be run easily and systematically for various reasons discussed later. The flexibility of the triggers in PostgreSQL can also be a valuable resource for triggering archiving tasks when the data becomes too large.

3.3.4 Database Design and Schema

Following the above analysis, the final database design is implemented in the relational database PostgreSQL, with data being stored and represented in tables. Catering to the needs of the product, the schema design is shown in Figure 3.

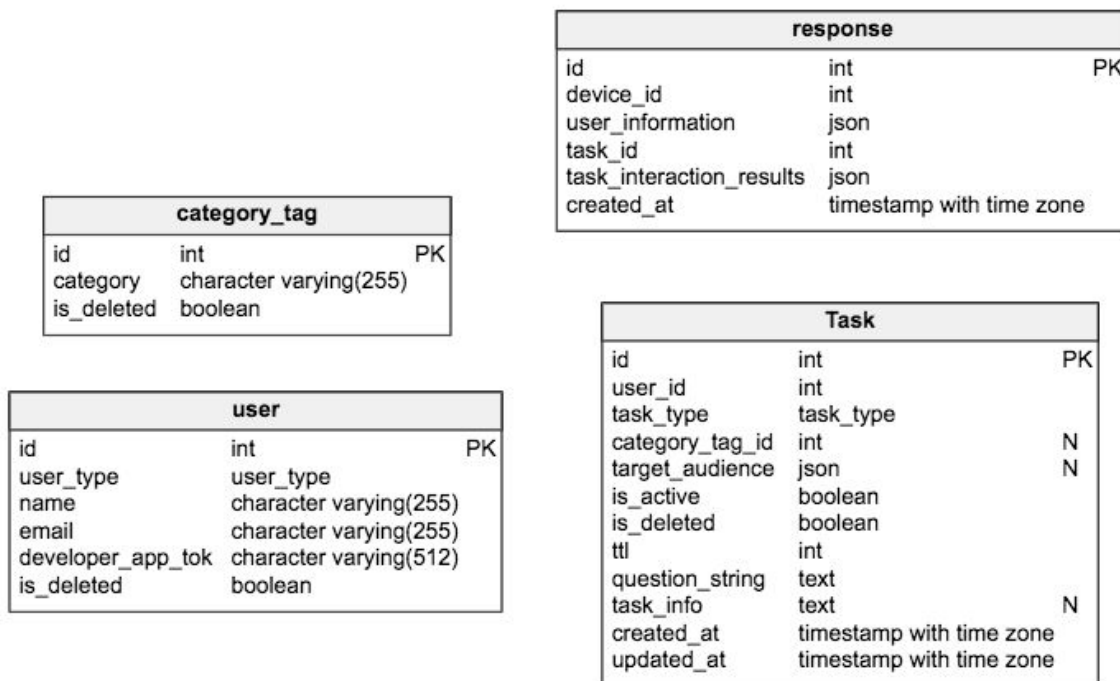


Figure 3. Schema of the database (one block per table and each field represents a column).

Each row in the task table encapsulates a single task which is used to display task information to mobile users. Noteworthy fields are category_tag_id, which corresponds to a category_tag in the category_tag table to use for tagging a task into a certain category such as health, sports, etc. Target audience is a **JavaScript Object Notation (JSON) blob** to allow for easy changes to be made to the features of a target audience classification. The time to live (TTL) field refers to how much time the task should be served for from the created_at date. There is a foreign key

from the `category_tag_id` to the `id` field of the `category_tag` table and another foreign key from the `user_id` table to the `id` field in the `user` table. A foreign key is simply a field in one table that uniquely identifies a row in another table.

Each row in the `user` table encapsulates a single user and provides all the information required for a user. The `user_type` determines whether the user is a developer or a researcher. The `developer_app_tok` is required for authentication reasons when trying to obtain a task from the database.

Each row in the `category_tag` table encapsulates a single category tag that a task can be tagged with. The purpose of the `category_tag` is to enable tasks to be targeted to certain demographics of app users.

Each row in the `response` table encapsulates a single response from an app user. Most of the task specific information is in the `task_interaction_results` field as a flexible JSON blob. This is the bulk of the information that is meant to be returned to the researcher. There is a foreign key from `task_id` to the `id` field in the `task` table.

3.3.5 Database Caching, Partitioning, and Archiving

Since Insightful generates large amounts of data, the database must be able to store all of the data efficiently for quick access. Thus, a cache containing the tasks that are still active (calculated by offsetting the TTL from the creation date) should be generated every day to allow for quick access to these active tasks.

The two most popular caching services are Redis and Memcached. Both offer significant improvements to speed in data retrieval operations in the form of in memory caches, so speed is not a contender when choosing between the two. The main appeal of Memcached is that it is simpler to setup and use for users who don't have very complex requirements such as monitoring and analytics. Redis, on the other hand, offers an array of built-in services such as monitoring, automatic persistence to disk, and transactions with optimistic locking. It can also often be used as a data structure server since it allows complex data types such as Hashes, Sorted Sets, and Lists whereas Memcached supports only key-value pair storage [19].

Monitoring queries to the cache can be useful to allow potential analytics information to both improve the cache design and also provide the researchers with feedback on issues such as their task serving rate. The support that Redis gives for more complex data types will also allow for a more efficient cache structure. For example, using a HashSet to store active tasks with the key as the category tag and values as a list of tasks would allow faster retrieval of tasks by category. Thus, Redis appears to be the best caching service for the Insightful platform.

Database **partitioning** simply means breaking a table down into smaller partitions of information that can be stored on different virtual blocks. This technique is always recommended for larger

tables and all the information in a row will always belong to the same partition. The partition key is defined as a column which will be used to determine the row's partition [20]. Partitioning is used by the database for the response table, since the response table will grow too quickly to be stored on the same partition. The partition key will be generated by date so that there is a roughly even distribution of entries in each partition.

For now, data archiving is done manually whenever is deemed necessary. When the database is too large, data is copied from the database onto a disk to be stored and wiped from the database to make room for newer entries.

3.4 Mobile Application Programming Interface (API) Design

3.4.1 API Architecture

The Insightful platform requires the creation of an API that powers the mobile software development kit and acts as an interface for pipelining data in and out of the database. In accordance with the specifications, this API supports endpoints that serve tasks and receive information from served tasks as well as communicating with the mobile SDK, database, and website. Since an API is functionally equivalent to a specialized web server, the API is implemented using the same architecture and technical tools outlined in section 3.2 for the web server. This means that the API runs on top of a technical layer consisting of Node.js, TypeScript, and Amazon's suite of web services. By utilizing the same technology stack for the mobile API as for the web server, developer productivity is boosted since new technologies do not have to be learned and the API's specification of having to be scalable is automatically fulfilled due to Amazon web service's autoscaling options.

3.4.2 Task Serving Load Balancing

In order to satisfy the API specifications of "the API must be able to manage a large number of servable tasks" and "the API must be able to serve tasks concurrently up to a limit", a task serving load balancing algorithm is required. To ensure fairness and that all researchers are treated equally, ideally all active tasks currently on the Insightful platform should be served the same amount of times. As was mentioned in section 3.3 about the database, every day a list of active tasks for that day is generated and then stored in a caching database in order to ensure quick task retrieval times.

One approach to load balancing would be simply choosing a random task from the daily generated list of active tasks every time a task is requested. This approach has the advantage of having no side effects and consequently, it is infinitely parallelizable and does not have to worry about any potential issues with regards to concurrent access to the caching database. This approach would also guarantee a uniform distribution of served tasks as long as there are sufficient task serving requests to smoothen out the trend of choosing a random task each time. However, this approach breaks down quickly when tasks of a certain category are requested

non-uniformly, leading to the potential of tasks in other categories suffering from starvation and never being served.

Another approach to load balancing would be to simply shuffle the active tasks of a day and placing them into a queue. When a task is requested, the queue is dequeued, the task is served, and then the task is enqueued to the back of the queue. This approach has the advantage of guaranteeing a uniform distribution of all tasks. However, it suffers from the computational complexity of shuffling the tasks and requires making the queue thread safe such that only a single task request can access the queue at a time (thus creating a performance bottleneck). This approach is also not able to prevent starvation due to tasks of a certain category being requested exclusively.

A third approach to load balancing would be to sort all active tasks by their category and then choosing a task randomly from a category when a task request arrives that specifies a category. If a task request does not specify a desired category, a random category is selected with weight given to categories that have been served less that day. This weight adjustment can be accomplished by performing **roulette wheel selection** used in genetic algorithms where the probability of a category being selected is inversely proportional to the percentage of total tasks that have been served that day from that category. [21] This is the equivalent of spinning a roulette wheel in a casino with areas one can bet on of varying sizes that consequently correspond to varying probabilities of being selected relative to the rest of the roulette wheel. Finally, in order to avoid the starvation of tasks (when tasks of only a subset of categories are requested), every time a task is requested, there is a small probability that the category of the task request is ignored and instead a random task from a different category is returned.

This final approach to load balancing has the downside of being more complicated to implement and not being able to guarantee a perfectly uniform distribution of tasks served. However, it has the advantage of being parallelizable, not performing any computationally expensive shuffles, and preventing the starvation of tasks due to certain categories being requested exclusively. As a result, this load balancing algorithm is further investigated, tested, and implemented.

3.4.3 Optimizing Load Balancing using a Monte Carlo Simulation

In order to determine the percentage of time the load balancing algorithm should ignore the category of a task request, numerical analysis must be conducted. This can be done using a **Monte Carlo simulation** which has the advantage of being computationally fast to conduct and can be run multiple times in order to retrieve statistically significant results. For this particular problem, the simulation consists of 300 tasks split into 3 different categories of 100 tasks each and 3 000 000 task requests being generated with varying levels of starvation. Ideally, each task should be served 10 000 times each if they are being uniformly distributed, however, as some categories begin to dominate, the task requests will become skewed towards those particular categories. However, the goal is to prevent starvation of tasks and each task should be served at least once if a proper percentage for the load balancing algorithm to ignore the

task request category is selected. Table 3 summarizes the results from running the Monte Carlo simulation for 1000 iterations for various different configurations of the load balancing algorithm.

Table 3. Results from running Monte Carlo simulations

Tasks	Categories	Requests	Starved Categories	Ignore Category Percentage	Minimum Requests per Task	Average Requests per Task
300	3	3 000 000	0	1%	9774	10 000
300	3	3 000 000	0	5%	9792	10 000
300	3	3 000 000	0	10%	9781	10 000
300	3	3 000 000	1	1%	91	10 000
300	3	3 000 000	1	5%	483	10 000
300	3	3 000 000	1	10%	967	10 000
300	3	3 000 000	2	1%	39	10 000
300	3	3 000 000	2	5%	232	10 000
300	3	3 000 000	2	10%	472	10 000

As can be seen from Table 3, even a category ignoring percentage as low as 1% still prevents the minimum requests per tasks from dropping to zero which would indicate that starvation is occurring and not all tasks are being given a chance to be served to a task request. As a result, the selected category ignoring percentage for the load balancing algorithm in a typical configuration can be 1% or even lower without reducing the minimum requests per tasks to zero. Naturally, this percentage will scale linearly with the number of tasks that are available to choose from and there must be a large number of task requests relative to the number of tasks available.

3.4.4 Verifying Load Balancing Optimization using Simulated Annealing

Simulated annealing is a metaheuristic algorithm that can be used to approximate the global optimization of a large search space. It works by simulating the physical annealing process by trying out many different random solutions to the problem in the beginning and then slowly “cooling” to an approximate optimal solution as the “temperature” of the annealing process decreases. [22] In the case of load balancing, it can be used to verify the results obtained from the previous section where the Monte Carlo simulation was brute forced over many different configurations and potential solutions. The cost function of the simulated annealing algorithm that it is attempting to optimize is getting the minimum requests per task as close to zero as possible without it ever being equal to zero. In the cost function, a value of zero is punished heavily since this indicates absolute starvation. Table 4 summarizes the results from applying simulated annealing to the various different configurations for the load balancing algorithm.

Table 4. Results from applying simulated annealing.

Tasks	Categories	Requests	Starved Categories	Simulated Annealing Ignore Category Percentage	Minimum Requests per Task	Average Requests per Task
300	3	3 000 000	0	No significant result.	9776	10 000
300	3	3 000 000	1	0.0124%	8	10 000
300	3	3 000 000	2	0.0203%	11	10 000

From Table 4, one can conclude that for this configuration of the problem, a very low ignore category percentage can be used for the load balancing algorithm (about ten times smaller than 1%). These results agree with the results from Table 3 that were found using the Monte Carlo simulation and consequently, simulated annealing effectively verifies the legitimacy of the brute-force Monte Carlo simulations over various different configurations.

3.5 Mobile Software Development Kit Design

Adoption of Insightful relies on developers that choose to incorporate the platform into their native mobile applications. Developers act as the bridging element between researchers and app users. Apart from monetary incentives, a series of requirements must be met to eliminate any potential burdens on the developers; as noted by the specifications in Section 2. The platform should be widely accessible. Developers must be able to easily integrate the Insightful platform without excessive effort or specialized knowledge on the internal workings of the platform. This means that any updates to the platform should include backwards compatibility so that developers do not need to modify their project with every iteration of the package including new bug fixes or feature updates. For example, developers using version xx.xx.032 of the platform should not be affected when version xx.xx.033 or above is released. At the same time, developers should be able to update to version xx.xx.033 or higher as soon as it is released should they choose to do so. With all this in mind, the proper design of a SDK must be taken into consideration to satisfy all the above criteria, in particularly a SDK that is designed for the mobile iOS platform.

3.5.1 Modularization in Software

The main purpose of a SDK is to provide a pre-packaged and modularized piece of software that performs a specialized set of tasks independently from the rest of the software project. Reasons for modularization include improving readability, simplifying debugging, and increasing reusability [23]. Lastly, functionality that has been broken down and packaged individually can

be reused in different parts of the application and can even be used in different parts of different applications. This is where the development of libraries and frameworks come into play.

Libraries and frameworks are both individual software projects, created to tackle a specific subproblem which can be shared between developers [24]. Without these external packages, application development time would increase. In other words, in a world where the concept of libraries and frameworks does not exist, every application developer that would like to include payment processing would need to implement their own architecture to handle payment information, payment verification, and security protocols. However, with the introduction of external libraries and frameworks, once a developer implements a solution, they are able to generalize it and share it with other developers for reuse. This enables other developers to spend less time implementing common modules and focusing on the unique functionalities of their project. For this reason, Insightful provides a mobile SDK. Since Libraries and Frameworks are written in native code [25], it is able to utilize native libraries that add responsiveness to the user interface. This accomplishes the specifications that the mobile SDK must be responsive as stated in Section 2.

An alternative to providing a mobile SDK would be to expose parts of the API to public developers as an endpoint to query for tasks and submit responses. The developers would then have to implement their own custom display components to surface the information visually to the application users. Although this offers more freedom of implementation to the developer, it would ultimately go against the ease of integration specification.

3.5.2 Libraries vs. Frameworks

While there are many ways to create a mobile SDK, static libraries and dynamic frameworks are the two patterns commonly followed in Apple's iOS environment. A static library is simply a package of **uncompiled source code** [24]. Developers using a library can dig into the code and understand the implementation from inside out. Since the code is uncompiled, developers also have the freedom and capability to modify the library to their liking. This allows for flexibility and customization, but also means that there is no proprietary value in the component since developers have access to the full implementation.

Frameworks, introduced in iOS 8.0, can be distributed as compiled binaries [25]. Compiled code is designed for computers to understand, but is not human readable. This means that without decompilation, a tedious and difficult task, a developer using the framework does not know its internal workings. Frameworks also have the added security of preventing malicious developers from exploiting the platform. This satisfies the specification of a secure design. Documentation is provided to let developers know, on a high-level, what functionalities the framework accomplishes and which of these functionalities are available to the developer. Since there is no requirement for the developer to understand the internal workings of the Insightful platform, and since proprietary information should be kept out of sight, it is preferable to move forward with a framework design. This decision is supported since Swift, the new iOS language, is only

supported by frameworks. As stated in the non-functional specifications section 2.2, the mobile SDK must support iOS 8.0 and higher. Since Swift Frameworks are the newest technology designed specifically for iOS 8.0, it will be more robust and less likely to cause integration issues in the future.

3.5.3 Dependency Managers: CocoaPods vs. Carthage

Distribution of the mobile SDK is another issue to tackle. The most primitive way to distribute any type of SDK would require hosting the framework binaries online and have it be publically available. The developers would then download the framework and manually incorporate it into their project. This includes using online repositories such as GitHub or BitBucket. These online repositories are a reputable source from which developers can download the framework onto their own development machines and then import the framework into their local projects. However, there are downsides to this approach as well.

Firstly, the process of manually downloading and importing the framework is quite cumbersome, especially if the original framework is subject to frequent updates. This means for every updated version of the framework, the developers must delete the old framework from their project and re-import the new version. Secondly, if projects need to be shared among developers working on the same application, having the imported framework embedded inside the project will increase the project size, which means the sharing of projects will be hindered by an unrelated framework. These problems are tackled by employing a dependency manager which standardises and simplifies the process for importing SDKs into a project.

Two popular Dependency Managers are CocoaPods and Carthage. CocoaPods works by allowing mobile SDK developers to upload their framework into a **Pod** which is publicly accessible [26]. Other developers wishing to integrate the mobile SDK simply create a podfile that defines which Pods they would like to include and which versions of the framework they would like to include and then CocoaPods handles importing and updating these frameworks automatically. Meanwhile, Carthage allows for the same functionality without the need of creating a centralized repository to store pods. While Carthage is simpler, it is not the industry standard and not as widely adopted as CocoaPods [27]. Seeing as wide outreach to developers is one of the main criteria in the specifications of the mobile SDK, CocoaPods provides a better solution than Carthage.

3.5.4 Architecture of the Insightful Framework

The mobile SDK has a single endpoint for the application developer to interact with. This endpoint is called by the developer using less than 5 lines of code to request a task to surface to the application user. When the application developer calls the endpoint, they temporarily pass control of the application to the framework [24]. This **inversion of control** is what allows the developer to forego any understanding of the underlying design of the framework. Using Apple's iOS environment, the framework collects a unique identifier from the user. This identifier is anonymous but allows Insightful to decide which tasks have already been shown to the user.

The Insightful framework presents the application user with a visual display of the simple task and collects the user's response. Once the response is collected, the framework creates a secure connection to the API and sends the response to the Insightful server to be persisted in the database. This satisfies the specification in section 2 of allowing developers to request and serve a task to the application user. On top of this, due to the nature of **object oriented programming** (OOP), developers are able to inherit the functionality of the framework and modify the UI elements to match the style and behaviour of the application which also satisfies the specification of allowing developers to integrate the tasks into their application seamlessly.

4 Discussion and Project Timeline

4.1 Evaluation of Final Design

The final design has five major subsystems that work together to meet the initial project objectives and specifications. The functional specifications and non-functional specifications, listed in Table 1 and Table 2, outline the mobile SDK, mobile API, database, website, and web server requirements that need to be met for the project.

The website and web server functional specifications are implemented such that researchers are able to sign up for a secure account and create tasks on Insightful. The website must allow the researchers to visualize and download the collected data in a secure manner that is resistant to any malicious attacks and malicious researchers trying to exploit the platform. The website subsystem meets the following objective: provide researchers with the ability to crowdsource tasks. This is accomplished by providing templates that can be used for creating such tasks. There is also added security of the researcher's information by encrypting passwords and using OAuth to verify the user.

The mobile SDK functional specifications include the ease of integrating the mobile SDK framework into an existing iOS application by adding no more than 5 lines of code, requesting unobtrusive tasks for the application, and being resistant to malicious developers. The mobile SDK design meets the objective of easily integrating the platform into an already existing application by putting all the logic of the mobile SDK into a complete, isolated package such that the developer only needs to import the mobile SDK and begin using it. The design also ensures that the application is secure by issuing tokens to each unique application such that they can be verified and malicious API calls can be prevented.

The database functional specification outlines that there shall be no duplication of data. The design of the database takes into account duplication of data by storing information in smaller tables such that the researcher's information is stored in one table and all related task information is stored in a different table. By storing only an identification token of the researcher for each task, a minimal amount of information is stored. This prevents duplication of data since

researcher's personal information does not need to be restored with each individual task: associating each researcher with their identification token is sufficient.

Lastly, the API specifications include having endpoints to communicate between the following subsystems: mobile SDK, database, and website. The API also receives information from the served tasks and only re-serves tasks after the time to live period expires. This is taken into account by the database design where it stores a TTL token and ensures the time expires of the token before re-serving the same task using the API. Lastly, the API design contains multiple endpoints for the project: one endpoint for the website and a separate endpoint for the mobile SDK. Both endpoints handle the passing of information throughout the entire system and in and out of the database.

Overall, all essential specifications are met with the final design. Some non-essential specifications have not been met yet since some of them (such as providing pre-computed statistics to researchers) require a lot of extra work to implement and will only be added to the system if time permits.

4.2 Use of Advanced Knowledge

The project uses many principles of upper year knowledge across all the Insightful subsystems. The topics that are studied in ECE 458, Computer Security, are used to prevent any possible attacks on system vulnerabilities. The design of Insightful uses Passport, discussed in Section 3.2.2, a module that utilizes many strategies such as OAuth to verify the user's identification to ensure the security of the system and prevent any malicious uses [11]. The passwords of users of the platform are encrypted using an open source library so that if the attacker obtains an encrypted password, they are not able to retrieve the original password. Having background knowledge about **SQL injection** attacks, **Cross-Site Request Forgery** (XSRF) attacks, and **Cross Site Scripting** (XSS) attacks also provides valuable information and context when designing the project. When the project subsystems are implemented it helps to consider potential threats and take appropriate measures to prevent them. Moreover, a secure application identification token is used to ensure that a task sent to the user's smartphone corresponds to the task response that is sent back from the smartphone.

Another course concept that is used is from Database Systems, ECE 356. When designing the database schema, a variety of data models are considered in order to store the information and represent it in tables. Database security and integrity is also strongly taken into consideration to ensure the privacy of each user. Performance characteristics of storage patterns are taken for account throughout the design of this subsystem.

Distributed Computing, ECE 454, knowledge is used in the design of the task distribution algorithm of the API. Techniques in concurrency and data synchronization are used extensively in order to implement the web server, API, and database. The course also provides a lot of

hands on knowledge with tools which directly aids in the architecture of the Insightful project; in one case from the course, full implementation of a distributed data store.

Lastly, Cooperative and Adaptive Algorithms, ECE 457A, knowledge is used when designing the API's load balancing algorithm. Concepts such as roulette wheel selection and simulated annealing from the course prove to be invaluable. As such, they are used to determine parameters of the load balancing algorithm for various scenarios and configurations.

4.3 Creativity, Novelty, Elegance

The novelty of this project stems from the idea of crowdsourcing tasks to gather information from diverse audiences. This is done by breaking down difficult problems into smaller, collective tasks and presenting them on smartphones by replacing ads. This cohesive system of surveying users and gathering insights for researchers provides a powerful tool for future research in medical, academic, and commercial fields. Insightful allows researchers to create anonymous profiles for users based on age, gender, and other features. This enables the platform to be the first tool that is designed to easily obtain longitudinal data: tracking the same user at different points in time and comparing their responses over a time period.

The elegance of the design is the simple to use and easy to integrate SDK. The simplicity of implementing the mobile SDK into the developer's application makes the product instantly gratifying for developers. This was designed with ease of integration in mind in order to increase the user base by encouraging more developers to use the platform.

4.4 Student Hours

Table 5 summarizes the total numbers of hours worked so far by the teammates on the Insightful project.

Table 5. Total hours worked per group member.

Student	Hours
	78
	78
	79
	81
	80

4.5 Potential Safety Hazards

Insightful is purely a software project and therefore there are no potential safety hazards. However, it does bring about potential concerns with respect to security and privacy. Software security is arguably one of the most important components when designing a software project. Applications that are prone to errors may result in variety of vulnerabilities ranging from compromising user privacy and account security to the execution of malicious attacks. Some of these attacks are mentioned in section 4.2 of this report, Use of Advanced Knowledge. This section also outlines what can be done to prevent these kinds of attacks and design the system in a secure way. Additionally, countermeasures can be taken to prevent such vulnerabilities by writing unit tests to continuously check the code for correctness, creating anonymous user profiles, and only storing essential information in the database.

4.6 Project Timeline

The project timeline for ECE498A and ECE 498B is presented via the Gantt chart in Figure 4 below. It breaks down the project into its major tasks and outlines the deadlines so that the final report, working prototype, and symposium presentations are completed on time.

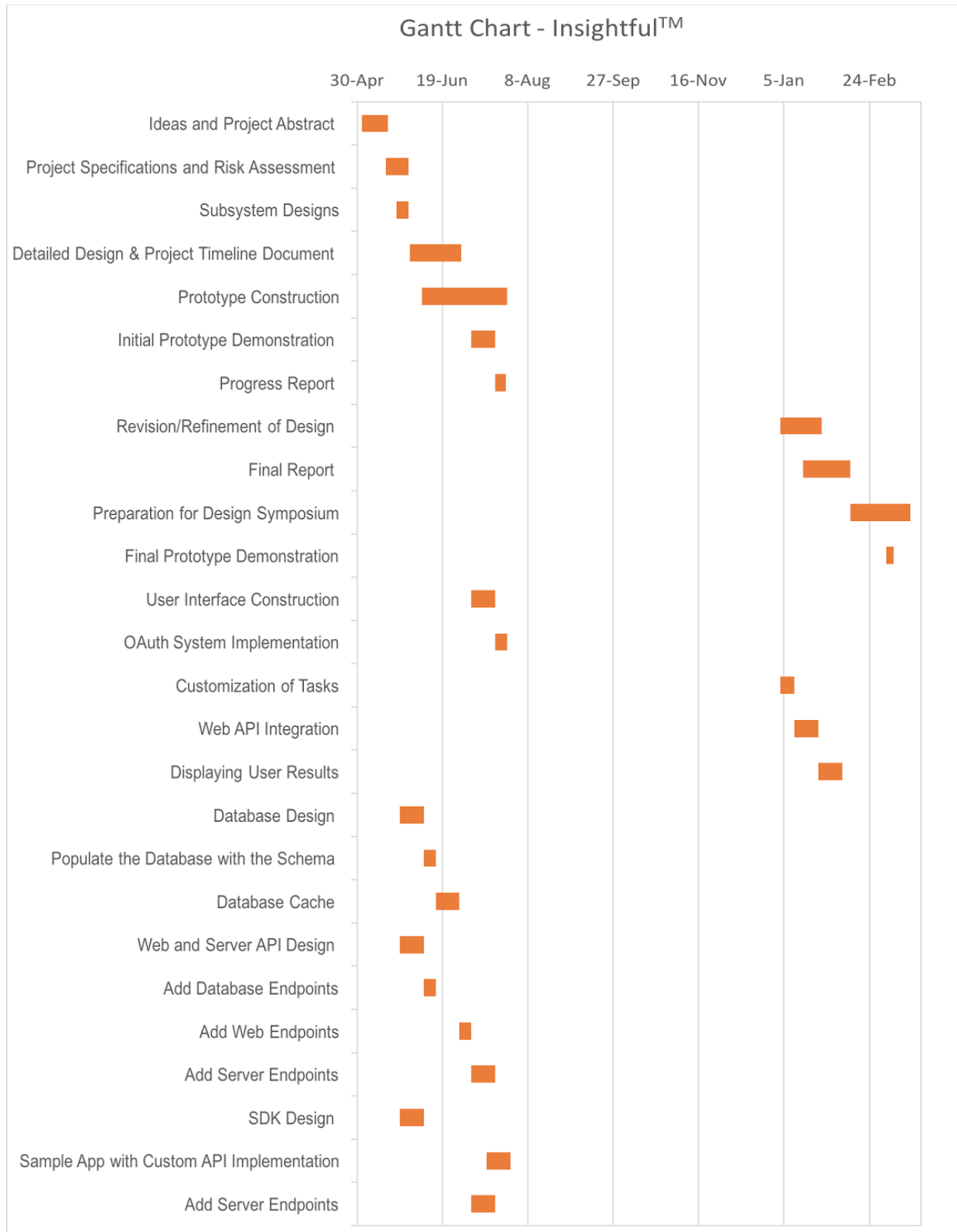


Figure 4. Project timeline shown with a Gantt chart.

5 Technical Terminology

Application Program Interface (API) is a set of routines, protocols and tools for building software applications. An API specifies how software components should interact.

Atomicity, Consistency, Isolation and Durability (ACID) is a set of properties that a reliable database transaction should have.

Auto Scaling is method used in cloud computing where the amount of computational resources in a server farm, typically measured in terms of the number of active servers which scale automatically based on the load on the farm.

Cascading Style Sheets (CSS) is a style sheet language used for describing the presentation of a document written in a markup language.

CloudFront is a global content delivery network (CDN) service that accelerates delivery of websites, APIs, video content or other web assets.

CocoaPod is a commonly used dependency manager. It allows developers to easily integrate third-party code into their own applications without manually copy and pasting source code.

Cross Site Request Forgery (XSRF) is a one click attack or session riding is a malicious exploit of a website where unauthorized commands are transmitted from a user that the website trusts.

Cross Site Scripting (XSS) is a type of computer security vulnerability typically found in web applications. It enables the attackers to inject client-side scripts into web pages viewed by other users. Such attack vulnerability may be used by attackers to bypass access controls such as the same-origin policy.

Domain Name System (DNS) is a hierarchical decentralized naming system for computers, services, or any resource connected to the Internet or a private network.

Elastic Cloud Compute (EC2) is a web service that provides resizable compute capacity in the cloud. It is designed to make web-scale cloud computing easier for developers.

Inversion of control in the context of this documentation represents the flow of control being passed from the developer to the framework. Instead of the developer telling the framework actions to complete, the developer allows the framework to assume temporary control to do whatever it needs, and then returning control back to the developer.

JavaScript is the programming language of HTML and the Web.

JavaScript Object Notation (JSON) blob is often referred to a JSON open-standard format that uses human-readable text to transmit data objects consisting of attribute-value pairs.

Model View Controller (MVC) is a software architectural pattern for implementing user interfaces on computers.

Monte Carlo Simulation is a broad class of computational algorithms that relies on repeated random sampling to obtain numerical results.

Multiversion Concurrency Control (MVCC) is a concurrency control method used by databases. Faster than the traditional way of using locks, it creates a snapshot for each user connected to the database, so any changes made by the writer will not be seen by readers until it is done.

Native in terms of software means that the subportion of code is written in the same development environment and language as its containing code.

Open Authentication (OAuth) is an open standard for authorization, commonly used as a way for Internet users to log in to third party websites using social accounts.

Object Oriented Programming (OOP) is a design model that revolves around objects instead of actions. Objects can be inherited, extended and subclassed.

Partitioning is a technique that allows tables and indexes to be divided into smaller pieces to enhance performance, manageability, and availability. These smaller pieces allow for easier access at a more granular level.

Passport.js is authentication middleware for Node.js

React is a JavaScript library for creating user interfaces

React Router is a complete routing library for React

Redux is a predictable state container for JavaScript apps.

Relational Database Management System (RDBMS) is a relational database model based on mathematical relations within sets of data.

Pod refers to a modular subunit of code that is part of the same Library or Framework project. Using CocoaPods, a developer can integrate as many Pods as they would like into their project.

Roulette Wheel Selection is a genetic operator used in genetic algorithms for selecting potentially useful solutions for recombination.

Simple Storage Service (S3) is an online file storage web service offered by Amazon

Simulated annealing is a method for solving unconstrained and bound-constrained optimization problems.

Software Development Kit (SDK) is typically a set of software development tools that allows the creation of applications for a certain software package, software framework, hardware platform, computer systems, operating systems or similar development platforms.

Stored procedures provide a way to batch SQL commands together and run them altogether either at certain scheduled times of the day, upon certain triggers, or manually.

Structured Query Language (SQL) is a language used to define and manipulate RDBMS databases.

SQL Injection is a code injection technique used to attack data-driven applications in which SQL statements are inserted into an entry field for execution which may allow the attacker to get the contents of the database, even the encrypted passwords.

Time to Live (TTL) is a mechanism that limits the lifespan or lifetime of data in a computer or database.

TypeScript is a strict superset of JavaScript, and adds optional static typing and class-based object-oriented programming to the language.

Uncompiled source code is a collection of human readable code written by a human developer to instruct the computer an action or a set of actions. The computer executes the code by first compiling the source code into compiled machine code, which can then be deciphered by a computer.

Uniform Resource Locator (URL) is a reference to a web resource that specifies its location on a computer network and a mechanism for retrieving it.

Unix is a family of multitasking, multi user computer operating systems

6 References

- [1] N. Folbre, "The Unregulated Work of Mechanical Turk", *Economix Blog*, 2016. [Online]. Available: <http://economix.blogs.nytimes.com/2013/03/18/the-unregulated-work-of-mechanical-turk/?smid=pl-share> [Accessed: 27- May- 2016].
- [2] A. Berinsky, G. Huber and G. Lenz, "Evaluating Online Labor Markets for Experimental Research: Amazon.com's Mechanical Turk", *Political Analysis*, vol. 20, no. 3, pp. 351-368, 2012.
- [3] J. Poushter, "Smartphone Ownership and Internet Usage Continues to Climb in Emerging Economies", *Pew Research Center's Global Attitudes Project*, 2016. [Online]. Available: <http://www.pewglobal.org/2016/02/22/smartphone-ownership-and-internet-usage-continues-to-climb-in-emerging-economies/>. [Accessed: 27- May- 2016].
- [4] J. Koetsier, "30 billion times a day, Google runs an ad (13 million times, it works)", *VentureBeat*, 2012. [Online]. Available: <http://venturebeat.com/2012/10/25/30-billion-times-a-day-google-runs-an-ad-13-million-times-it-works/>. [Accessed: 27- May- 2016].
- [5] "MVC Architecture," - *Google Chrome*. [Online]. Available: https://developer.chrome.com/apps/app_frameworks. [Accessed: 30-Jun-2016].
- [6] P. Selle, T. Ruffles, C. Hiller, and J. White, *Choosing a JavaScript framework*. Santa Rosa (Calif.): Bleeding edge press, 2014.
- [7] "A JavaScript library for building user interfaces | React," *A JavaScript library for building user interfaces | React*. [Online]. Available: <https://facebook.github.io/react>. [Accessed: 30-Jun-2016].
- [8] "TypeScript," *Wikipedia*. [Online]. Available: <https://en.wikipedia.org/wiki/typescript>. [Accessed: 30-Jun-2016].
- [9] "Redux," *Introduction*. [Online]. Available: <http://redux.js.org/docs/introduction/>. [Accessed: 30-Jun-2016].
- [10] "Adzerk Blog," The Fastest Ad Server In The World Just Got Even Faster. [Online]. Available: <http://adzerk.com/blog/2013/03/the-fastest-ad-server-in-the-world-just-got-even-faster/>. [Accessed: 30-Jun-2016].
- [11] "Passport," *Documentation*. [Online]. Available: <http://passportjs.org/docs>. [Accessed: 30-Jun-2016].
- [12] "Choosing your Node.js Authentication Strategy," *Stormpath User Identity API*, 2014. [Online]. Available: <https://stormpath.com/blog/choosing-nodejs-authentication-strategy>. [Accessed: 30-Jun-2016].
- [13] "AWS | Application Architecture Center," *Amazon Web Services, Inc.* [Online]. Available: <https://aws.amazon.com/architecture>. [Accessed: 30-Jun-2016].

- [14] "SQL RDBMS Concepts", *www.tutorialspoint.com*, 2016. [Online]. Available: <http://www.tutorialspoint.com/sql/sql-rdbms-concepts.htm>. [Accessed: 28- Jun- 2016].
- [15] "SQL", *Wikipedia*, 2016. [Online]. Available: <https://en.wikipedia.org/wiki/SQL>. [Accessed: 28- Jun- 2016].
- [16] L. Issac, "SQL vs NoSQL Database Differences Explained with few Example DB", *Thegeekstuff.com*, 2016. [Online]. Available: <http://www.thegeekstuff.com/2014/01/sql-vs-nosql-db/>. [Accessed: 28- Jun- 2016].
- [17] E. McNulty, "SQL vs. NoSQL- What You Need to Know - Dataconomy", *Dataconomy*, 2014. [Online]. Available: <http://dataconomy.com/sql-vs-nosql-need-know/>. [Accessed: 28- Jun- 2016].
- [18] O. Tezer, "SQLite vs MySQL vs PostgreSQL: A Comparison Of Relational Database Management Systems | DigitalOcean", *Digitalocean.com*, 2016. [Online]. Available: <https://www.digitalocean.com/community/tutorials/sqlite-vs-mysql-vs-postgresql-a-comparison-of-relational-database-management-systems>. [Accessed: 28- Jun- 2016].
- [19] C. Zulauf, "Memcached vs. Redis?", *Stackoverflow.com*, 2016. [Online]. Available: <http://stackoverflow.com/questions/10558465/memcached-vs-redis>. [Accessed: 28- Jun- 2016].
- [20] "Partitioning Concepts", *Docs.oracle.com*, 2016. [Online]. Available: https://docs.oracle.com/cd/B28359_01/server.111/b32024/partition.htm. [Accessed: 28- Jun- 2016].
- [21] "Fitness proportionate selection", *Wikipedia*, 2016. [Online]. Available: https://en.wikipedia.org/wiki/Fitness_proportionate_selection. [Accessed: 30- Jun- 2016].
- [22] "Simulated Annealing -- from Wolfram MathWorld", *Mathworld.wolfram.com*, 2016. [Online]. Available: <http://mathworld.wolfram.com/SimulatedAnnealing.html>. [Accessed: 30- Jun- 2016].
- [23] "Creating Static Library in iOS App Development | TechNet", *TechNetExperts*, 2016. [Online]. Available: <http://www.technetexperts.com/mobile/creating-static-library-in-ios-app-development/>. [Accessed: 30- Jun- 2016].
- [24] "Framework vs Library (Static & Shared) in Cocoa, Cocoa Touch", *Knowstack.com*, 2015. [Online]. Available: <http://www.knowstack.com/framework-vs-library-cocoa-ios/>. [Accessed: 30- Jun- 2016].
- [25] "Creating a Framework", *Developer.apple.com*, 2016. [Online]. Available: <https://developer.apple.com/library/mac/documentation/MacOSX/Conceptual/BPFrameworks/Tasks/CreatingFrameworks.html>. [Accessed: 30- Jun- 2016].
- [26] J. Greene, "How to Create a CocoaPod in Swift", *Ray Wenderlich*, 2015. [Online]. Available: <https://www.raywenderlich.com/99386/create-cocoapod-swift>. [Accessed: 30- Jun- 2016].
- [27] J. Frost, "Carthage Tutorial: Getting Started", *Ray Wenderlich*, 2015. [Online]. Available: <https://www.raywenderlich.com/109330/carthage-tutorial-getting-started>. [Accessed: 30- Jun- 2016].